

# Atlas-MAG Graph Audit

Correct Math, Wrong Ontology:  
How the Graph Catches What Testing Cannot

NestedLearning Project  
HADES\_DATABASE=NL

February 2026

## 1 Executive Summary

The Atlas-MAG\_OmegaRule implementation (4 months, 4,805 lines, 109 tests, 85.9% NIAH accuracy) was built *without* the NL graph methodology. Post-hoc audit against the HADES graph reveals a consistent pattern:

**The mathematical layer is correct.** Every equation—Omega loss (Eq 9), momentum (Eq 32), NS-5 update (Eq 33), polynomial features (Eqs 4–5), MAG gating—is faithfully implemented.

**The architectural layer is wrong.** The code introduces structural concepts (separate memory modules, training/inference mode distinction, multiple external APIs, invented persistent memory) that do not exist in the paper’s ontology.

**All 109 tests pass.** Tests verify math. The graph verifies ontology. These are orthogonal.

This report documents the specific structural breaks with graph diagrams suitable for inclusion in the methodology article.

## 2 Figure 1: The Two-Layer Audit

The graph enforces compliance at two layers: **equations** (mathematical correctness) and **code smells** (structural correctness). The old implementation passes the first and fails the second.

## 3 Figure 2: The Ontological Break

The core of the problem: the code introduces **architectural concepts that do not exist in the paper**. These invented abstractions create structural coupling that the paper’s design does not require.

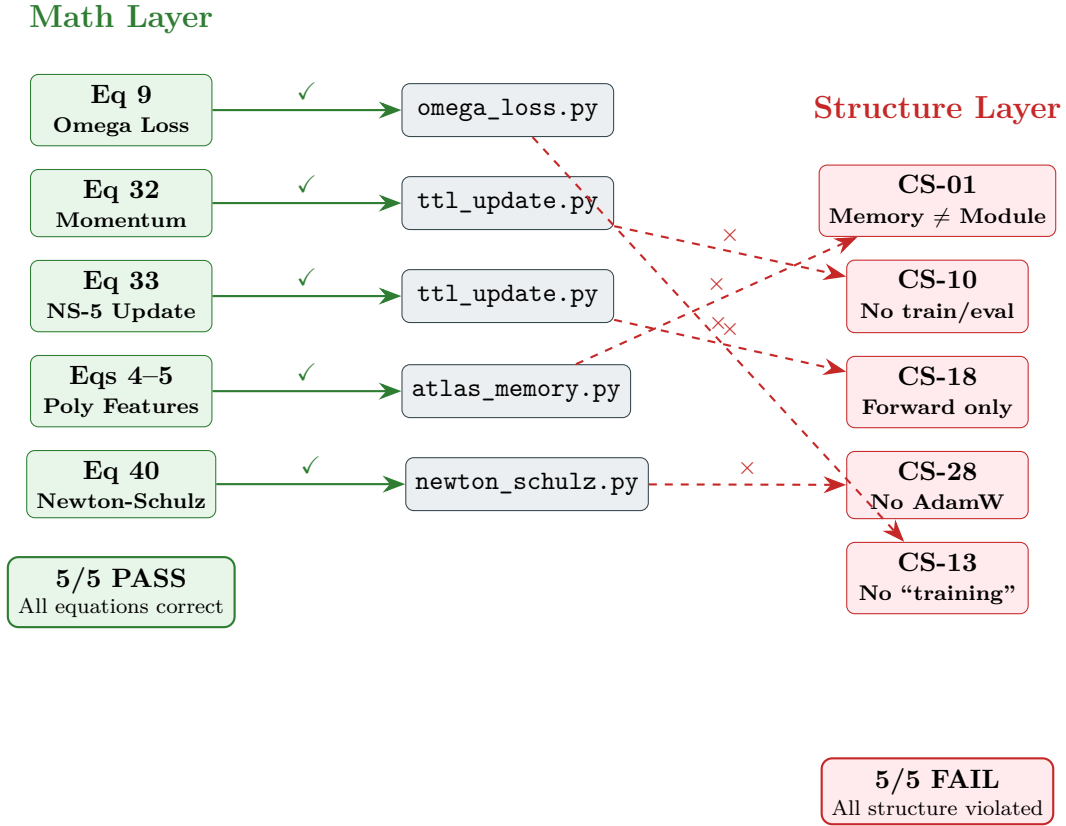


Figure 1: **Two-Layer Audit.** The mathematical layer (left, green) traces correctly from every paper equation to its code implementation. The structural layer (right, red) flags violations at every code module. Tests verify the green arrows. Only the graph verifies the red arrows.

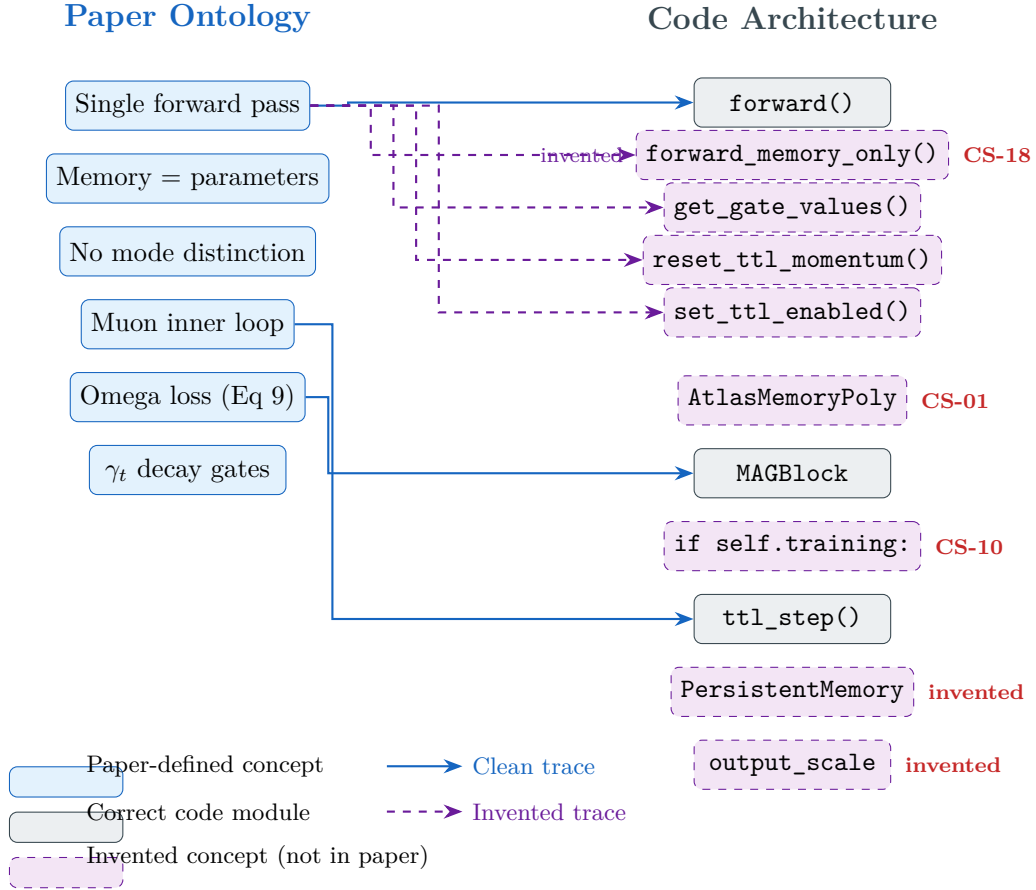


Figure 2: **The Ontological Break.** Left: what the paper defines. Right: what the code built. Solid blue arrows trace correctly from paper to code. Dashed purple nodes are concepts the code *invented* that have no paper origin: 4 extra API methods, a separate memory module class, a training mode gate, persistent memory, and a learnable output scale. The graph’s code smell rules (CS-01, CS-10, CS-18) flag each invention.

## 4 Figure 3: The Behavioral Consequence

Not all structural violations are cosmetic. One violation—CS-10, the `self.training` gate—changes the model’s *behavior* by disabling test-time learning during inference, which defeats the paper’s core premise.

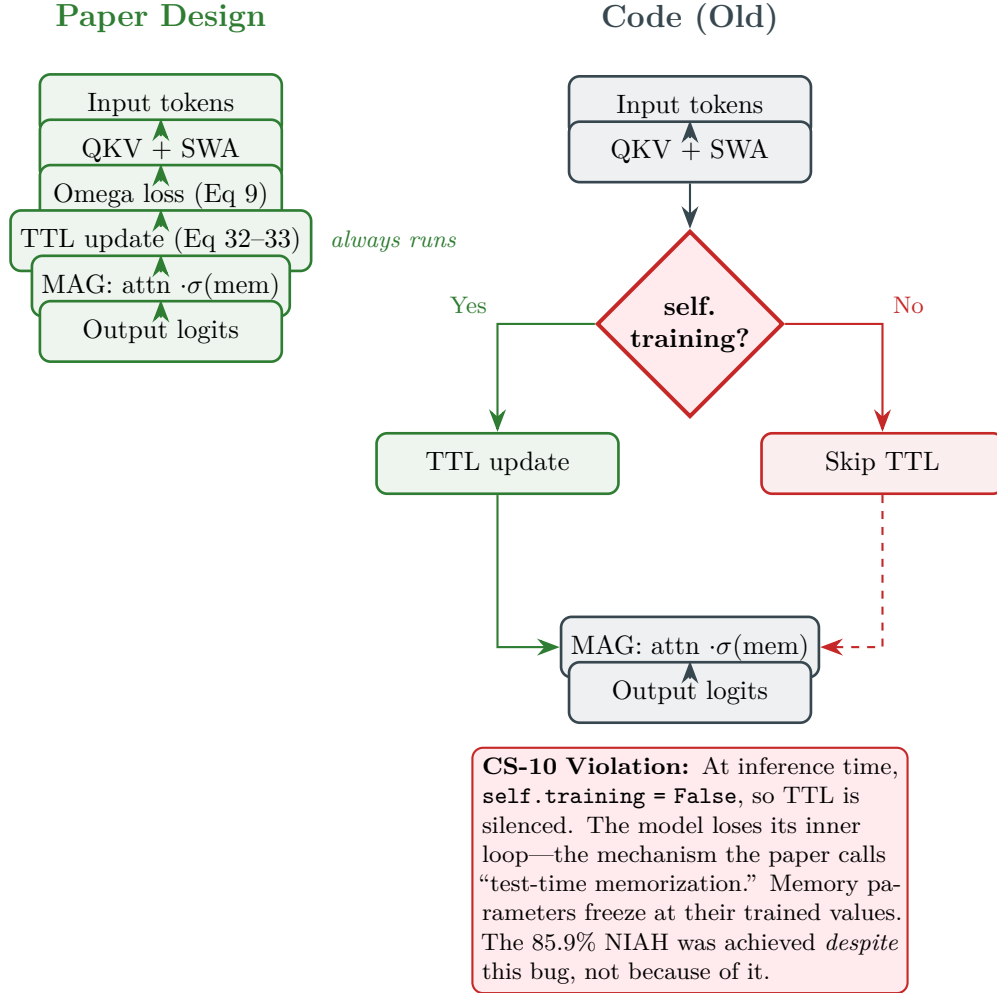


Figure 3: **Behavioral Consequence of CS-10.** The paper’s design (left) runs TTL on every forward pass—training and inference alike. The code (right) gates TTL behind `self.training`, disabling it during inference. This single structural violation changes the model’s behavior at test time. Tests cannot catch this because tests run in training mode.

## 5 Figure 4: Equation Coverage vs. Structural Compliance

This figure shows the key asymmetry: perfect equation coverage coexists with pervasive structural violation.

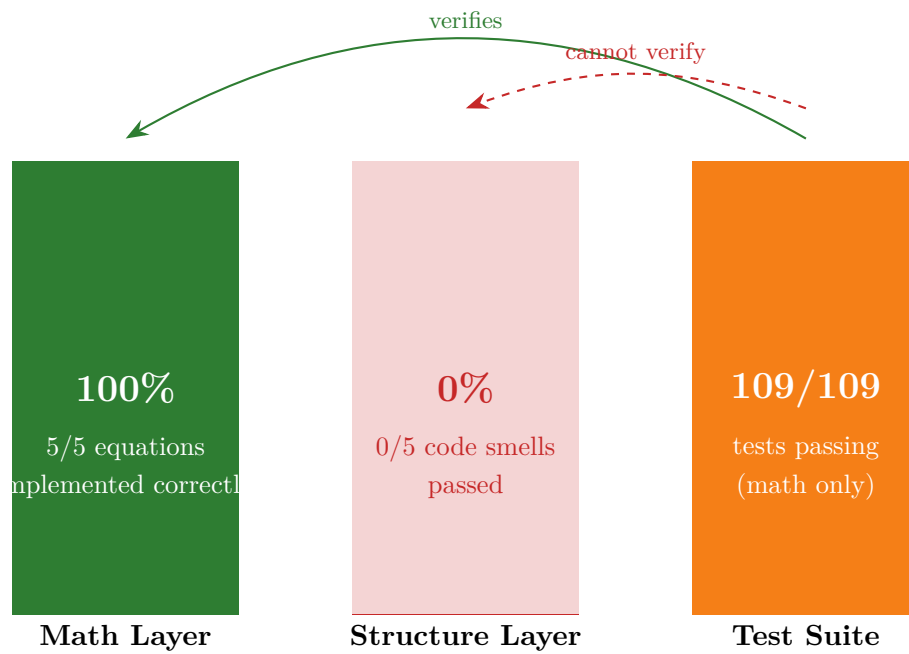


Figure 4: **The Coverage Gap.** The test suite verifies mathematical correctness (green bar) but has no mechanism to verify structural compliance (red bar). All 109 tests pass because they test equations, not ontology. The graph fills this gap—code smell rules operate at the structural level that tests cannot reach.

## 6 Figure 5: The Graph as Immune System

When code passes through the graph’s code smell rules, structural violations are detected before they propagate into the architecture.

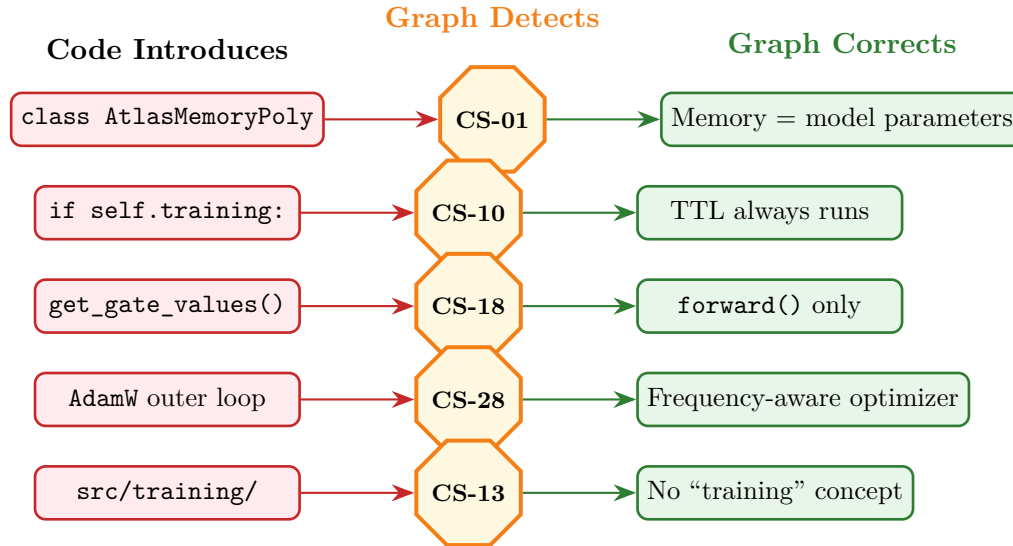


Figure 5: **The Graph as Immune System.** Each code smell rule (octagon) detects a specific class of structural violation. Violations enter from the left (red); the graph rejects them and prescribes corrections (green, right). Without the graph, all five violations passed through 109 tests undetected.

## 7 Detailed Violation Catalog

## 8 Invented Concepts

Beyond code smell violations, the implementation introduces concepts with **no paper origin**:

## 9 What Tests Cannot Catch

The fundamental insight of this audit:

*Tests verify that code computes the right answer.  
The graph verifies that code asks the right question.*

All 109 tests verify mathematical properties:

- Newton-Schulz produces near-orthogonal matrices ( $\|XX^T - I\|_F < \epsilon$ )
- Polynomial features have correct output dimension ( $d_k + \binom{d_k+1}{2}$ )
- Omega loss decreases with more context tokens
- TTL step reduces memory prediction error
- MAG gating is bounded in  $[0, 1]$

None of the 109 tests verify structural properties:

- Does TTL run during inference? (No—`self.training` blocks it)
- Is memory a separate module? (Yes—violates CS-01)

| Rule  | File            | Violation                                                                               | Paper Says                                                                 | Severity        |
|-------|-----------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------|
| CS-01 | atlas_memory.py | Memory is a separate <code>nn.Module</code> with own <code>forward()</code>             | Memory IS the parameters of the MLP; no separate module boundary           | High            |
| CS-10 | blocks.py:245   | <code>if self.training:</code> gates TTL                                                | No mode distinction; TTL runs on every forward pass (“test-time” learning) | <b>Critical</b> |
| CS-13 | src/training/   | Directory named “training”; concepts use training terminology                           | Context processing is continuous; “training” implies a phase that ends     | Low             |
| CS-18 | skeleton.py     | 5 extra methods: <code>forward_memory_only</code> , <code>get_gate_values</code> , etc. | Forward pass IS the only external API                                      | Medium          |
| CS-28 | train.py        | AdamW used for outer loop optimization                                                  | Outer loop must be frequency-aware; Muon/M3 family                         | Medium          |

Table 1: Complete catalog of structural violations detected by graph audit. CS-10 is the only violation that changes model *behavior*; the others affect code organization and conceptual clarity.

| Invented Concept | What It Does                                                       | Why It’s Wrong                                                                                                   |
|------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| PersistentMemory | Precomputed $M_{\text{persistent}}$ from “learned persistent keys” | Paper has no persistent memory for Omega Rule. Correct init is $M_0 = 0$ .                                       |
| output_scale     | Learnable scalar (init 0.15) to match attention magnitude          | Paper’s MAG uses $\sigma(\text{mem})$ which already bounds to $[0, 1]$ . Extra scale distorts the gating signal. |
| TTLUpdater class | Stateful wrapper with reset modes and step counting                | The inner loop is a mathematical operation (Eqs 32–33), not a stateful object with lifecycle methods.            |
| poly_norm        | LayerNorm on polynomial features before MLP                        | Paper specifies $\varphi(x) = [x, x_i x_j]$ directly into MLP. Normalization changes the feature distribution.   |

Table 2: Concepts invented during implementation that have no basis in the paper. Each represents architectural drift—a small decision that compounds over 4 months of development.

- Does the model expose multiple APIs? (Yes—5 extra methods beyond `forward`)
- Are invented concepts present? (Yes—persistent memory, output scale)

The graph’s code smell rules are the **structural type system** that tests cannot replace.

## 10 Conclusion

The Atlas-MAG implementation demonstrates that **mathematical correctness is necessary but not sufficient**. A codebase can implement every equation faithfully, pass every test, and still

violate the paper’s design at the structural level.

The NL graph methodology addresses this gap through:

1. **Equation nodes** that ensure mathematical traceability
2. **Code smell rules** that enforce structural compliance
3. **IS/IS NOT axiom containers** that reject foreign concepts
4. **Pseudocode generation** from graph nodes (not from human interpretation)

The old Atlas-MAG code is now archived as a historical artifact—a controlled example of what paper-to-code looks like *without* the graph. The new pseudocode (5 tiers, 38 equations traced) shows what the graph *produces*. The contrast between the two is the methodology’s evidence.

---

**Graph source:** HADES\_DATABASE=NL, collections: atlas\_equations (60), atlas\_axioms (2), nl\_code\_smells (36)

**Old code:** [github.com/toddwbucy/Atlas-MAG\\_OmegaRule](https://github.com/toddwbucy/Atlas-MAG_OmegaRule) (archived)

**New pseudocode:** [github.com/toddwbucy/Atlas](https://github.com/toddwbucy/Atlas) (5 tiers)

**Generated:** February 10, 2026